

DSA & Advanced DSA Roadmap

Phase 6 — Hashing

Learning Objectives

By the end of this phase, learners will be able to:

- Explain how a hash function maps keys to bucket indices, and what makes a hash function 'good' (uniform distribution, determinism, low collision rate).
- Implement collision handling using both chaining and open addressing.
- Explain load factor and rehashing, and why a hash table resizes once it grows too full.
- Identify and apply core hashing patterns: frequency counting at scale, pair-sum lookups, prefix hashing, hash combined with sliding window, and set-based membership problems.
- Solve Easy to Medium level hashing problems independently, applying prefix-hash and hash-plus-sliding-window techniques to problems that would otherwise require brute-force scanning.

Implementation & Internals

Core Concepts

- Hash Functions — mapping arbitrary keys to integer indices; properties of a good hash function (deterministic, fast, uniform).
- Buckets — the array slots a hash table distributes keys into, and how multiple keys can land in the same bucket.
- Collision Handling — chaining (each bucket holds a list) versus open addressing (probing for the next free slot).
- Rehashing — growing the underlying array and redistributing all existing keys once the table becomes too full.
- Load Factor — the ratio of stored elements to bucket count, and why keeping it below a threshold preserves $O(1)$ average-case operations.

Implementation Exercises

- Build a hash map from scratch using an array of buckets, each bucket a simple list, implementing put, get, and remove (chaining).
- Add an automatic rehash step that doubles the bucket array once load factor exceeds a threshold (e.g. 0.75), and redistributes all existing keys.
- Implement a second version using open addressing with linear probing, and compare its behavior under heavy collisions to the chaining version.
- Write a simple string hash function (e.g. polynomial rolling hash) and demonstrate two different strings colliding to the same bucket index on a small table.

Pattern Mastery

Pattern 1: Frequency Counting

Concepts

- Counting occurrences using a hash map for arbitrary (non-array-index) keys, contrasted with the fixed-size frequency arrays used in Phase 1.
- Using frequency counts combined with a heap or bucket sort to answer 'top K' style queries.

- Recognizing when a hash set is sufficient (membership only) versus when a hash map is required (membership plus count).

Practice Problems

- Given an array, return the k most frequent elements.
- Given an unsorted array, find the length of the longest run of consecutive integers, using a hash set for $O(1)$ membership checks rather than sorting.

LeetCode

- [Top K Frequent Elements — leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)
- [Longest Consecutive Sequence — leetcode.com/problems/longest-consecutive-sequence](https://leetcode.com/problems/longest-consecutive-sequence)

Note: distinct from Phase 1's Frequency Counting (which used fixed-size arrays for bounded inputs) and Phase 2's Character Frequency (string-specific). This pattern emphasizes general hash-map-backed counting for arbitrary key types.

Pattern 2: Pair Sum

Concepts

- Using a hash map to record values seen so far and check, in $O(1)$, whether the complement needed to reach a target has already appeared.
- Extending the single-pass complement lookup to track positions, not just presence, when index distance matters.

Practice Problems

- Given an array and a target, find the indices of two numbers that sum to the target, in a single pass.
- Given an array and an integer k, determine whether any two equal values occur within distance k of each other.

LeetCode

- [Two Sum — leetcode.com/problems/two-sum](https://leetcode.com/problems/two-sum)
- [Contains Duplicate II — leetcode.com/problems/contains-duplicate-ii](https://leetcode.com/problems/contains-duplicate-ii)

Note: distinct from Phase 1's Two Pointers approach to pair-sum-style problems (which required a sorted array). This pattern uses a hash map and works on unsorted input in a single pass.

Pattern 3: Prefix Hashing

Concepts

- Storing the frequency of each prefix-sum remainder (or prefix sum itself) in a hash map as you scan, rather than recomputing sums from scratch.
- Recognizing that two prefix sums sharing the same remainder modulo k implies the subarray between them is divisible by k.

Practice Problems

- Count the number of subarrays whose sum is divisible by a given k.
- Given an array and k, determine whether there exists a subarray of length at least 2 whose sum is a multiple of k.

LeetCode

- [Subarray Sums Divisible by K — leetcode.com/problems/subarray-sums-divisible-by-k](https://leetcode.com/problems/subarray-sums-divisible-by-k)
- [Continuous Subarray Sum — leetcode.com/problems/continuous-subarray-sum](https://leetcode.com/problems/continuous-subarray-sum)

Note: distinct from Phase 1's Prefix Sum pattern (plain running totals via an array). This pattern combines prefix sums with a hash map to answer counting/existence questions in $O(n)$.

Pattern 4: Hash + Sliding Window

Concepts

- Maintaining a hash map of character or element frequencies inside a sliding window, updating it incrementally as the window moves.
- Shrinking the window when a hash-map-tracked constraint (e.g. too many distinct characters) is violated, then resuming growth.

Practice Problems

- Find the length of the longest substring that contains at most k distinct characters.

LeetCode

- [Longest Substring with At Most K Distinct Characters — leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters](https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters)

Note: distinct from Phase 1's and Phase 2's Sliding Window patterns, which tracked sums or fixed-size frequency arrays. Here the window's constraint is the size of a hash map itself (number of distinct keys).

Pattern 5: Set-Based Problems

Concepts

- Implementing a hash set from scratch to understand what a 'set' actually provides beyond a hash map: $O(1)$ membership with no associated value.
- Using a hash map to back a simplified hash map data structure, contrasted with the bucket-array implementation built earlier this phase.

Practice Problems

- Design and implement a HashSet supporting add, remove, and contains.
- Design and implement a HashMap supporting put, get, and remove, without using a built-in hash map.

LeetCode

- [Design HashSet — leetcode.com/problems/design-hashset](https://leetcode.com/problems/design-hashset)
- [Design HashMap — leetcode.com/problems/design-hashmap](https://leetcode.com/problems/design-hashmap)

Phase Assessment

Implementation Tasks

- Build a hash map from scratch using chaining, including an automatic rehash step.
- Build a second version using open addressing, and compare collision behavior between the two under a stress test of many inserts.
- Analyze and explain why average-case $O(1)$ lookup depends on keeping load factor bounded, and what happens to performance if rehashing is removed.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/two-sum
- leetcode.com/problems/contains-duplicate-ii
- leetcode.com/problems/design-hashset
- leetcode.com/problems/design-hashmap

Medium Problems

Frequency Counting

- leetcode.com/problems/top-k-frequent-elements
- leetcode.com/problems/longest-consecutive-sequence

Prefix Hashing

- leetcode.com/problems/subarray-sums-divisible-by-k
- leetcode.com/problems/continuous-subarray-sum

Hash + Sliding Window

- leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters

On scope and dedup

This phase has 9 distinct problems, larger than the Phase 1–3 baseline, and intentionally has no Hard tier: the natural Hard-difficulty hashing problems (e.g. Substring with Concatenation of All Words, Minimum Window Substring) either depend on patterns not yet covered or substantially overlap with sliding-window problems better suited to Phase 1–2. None of the 9 problems here repeats any problem from Phases 0–5, even though the pattern names 'Frequency Counting' and 'Pair Sum' are shared with Phases 1 and 2 — every problem under those names here is hash-map-specific and distinct from the array/string versions used earlier.

Coding Challenges (Assessment Day)

- Two Sum
- Top K Frequent Elements
- Longest Consecutive Sequence
- Subarray Sums Divisible by K
- Design HashMap

Expected Outcome

Students should be able to:

- Implement a hash map from scratch, including collision handling and rehashing.
- Recognize common hashing patterns: frequency counting, pair-sum lookups, prefix hashing, hash-backed sliding windows, and set-based membership checks.
- Solve unseen hashing problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium hashing questions independently, and explain the performance trade-offs between chaining and open addressing.